

ONLINE PAYMENT FRAUD DETECTION SYSTEM USING MACHINE LEARNING

**A Real-Time Adaptive Risk Scoring, Pre-Settlement Multi-Factor Security, and
Explainable AI FinTech Platform**

A Technical Project Report

Submitted in partial fulfillment of the requirements for the Degree of
Bachelor of Technology in Computer Science & Engineering

Candidate / Author: Afzal (GitHub: @Afzal006)

Repository: <https://github.com/Afzal006/Online-Payment-Fraud-Detection-System>

Domain: Financial Technology (FinTech) / Applied Machine Learning / Cyber-Defense

Dataset: PaySim Mobile Money Financial Fraud Synthetic Benchmark

Verified Test Suite: 502 Automated Unit, Integration & Security Tests (100% Passing)

Academic Year: 2025–2026



Scan QR for GitHub Repository Source Code & CI Artifacts

CERTIFICATE OF AUTHENTICITY

This is to certify that the technical project report entitled "**Online Payment Fraud Detection System Using Machine Learning**" is a bonafide record of authentic engineering research, architecture design, and software implementation carried out by **Afzal** under supervision. The software artifacts, machine learning pipelines, explainable AI integrations, and security enforcement mechanisms described herein have been verified against active codebases and **502 passing automated test cases**.

Internal Faculty Supervisor

Department of Computer Science

Signature: _____

Date: _____

Head of Department

Department of Computer Science

Signature: _____

Date: _____

DECLARATION OF ORIGINALITY

I hereby declare that this project report entitled "**Online Payment Fraud Detection System Using Machine Learning**" represents my original work. All external libraries (Scikit-Learn, XGBoost, SHAP, Flask, SQLAlchemy), reference datasets (PaySim Financial Dataset), and scientific publications have been appropriately cited. No fabricated survey results, fake credentials, or exaggerated metrics have been incorporated.

ACKNOWLEDGEMENTS

I express my profound gratitude to our academic mentors, faculty advisors, and the open-source software engineering community. Their continuous feedback, architectural guidelines, and scientific tooling made it possible to construct, evaluate, and formally validate this real-time financial fraud defense system.

EVALUATOR / FACULTY REMARKS:

Page 2 of 34

Evaluator Signature & Date: _____

TABLE OF CONTENTS & REPORT STRUCTURE

Section Title	Page	Section Title	Page
1. Abstract	Page 4	12. Implementation / Working Principle	Page 28
2. Introduction	Page 5	13. Testing & Validation (502 Tests)	Page 31
3. Problem Identification	Page 7	14. Results & System Screenshots	Page 34
4. Empathize & Define (Design Thinking)	Page 9	15. Core Functionality Code	Page 37
5. Ideation & Decision Matrix	Page 11	16. Project Evaluation & Metrics	Page 40
6. Requirements Analysis	Page 13	17. Conclusion & Learning Outcomes	Page 42
7. Technology Stack Rationale	Page 15	18. Future Enhancements & Roadmap	Page 43
8. System Design & DFDs (0, 1, 2)	Page 17	19. Project Links & QR Codes	Page 44
9. Database Design (11 Schemas)	Page 20	20. References & Bibliography	Page 45
10. Module Description (10 Modules)	Page 23	21. Appendix (APIs, DDLs, Secrets Template)	Page 46
11. UML Modeling (6 Diagrams)	Page 26		

LIST OF ABBREVIATIONS & ACRONYMS

Acronym	Full Description	Acronym	Full Description
UPI	Unified Payments Interface	SHAP	SHapley Additive exPlanations
ML	Machine Learning	XAI	Explainable Artificial Intelligence
SOC	Security Operations Center	PR-AUC	Precision-Recall Area Under Curve
OTP	One-Time Password	ROC-AUC	Receiver Operating Characteristic AUC
RBAC	Role-Based Access Control	IDOR	Insecure Direct Object Reference
ORM	Object-Relational Mapping	DFD	Data Flow Diagram
ERD	Entity-Relationship Diagram	MFA	Multi-Factor Authentication
P2P	Peer-to-Peer Transfer	WSGI	Web Server Gateway Interface

EVALUATOR / FACULTY REMARKS:

Evaluator Signature & Date: _____

1. Abstract

The rapid global expansion of instant digital payment networks—such as Unified Payments Interface (UPI), peer-to-peer (P2P) transfers, and digital banking rails—has revolutionized consumer commerce while simultaneously introducing severe vulnerabilities to sophisticated financial cyber-fraud syndicates. Conventional fraud prevention infrastructure relies almost exclusively on static, heuristic rule engines that suffer from excessive false positive rates and an inability to detect novel, complex account-draining attack vectors.

This technical project presents **FraudShield AI**, an enterprise-grade, end-to-end intelligent payment fraud detection, explainable risk assessment, and adaptive multi-factor verification platform. The proposed architecture integrates supervised machine learning classification using a tuned Random Forest ensemble ($F1 = 0.9985$, Precision = 1.0000, Recall = 0.9970 on PaySim test partitions) with real-time behavioral telemetry, including account-draining ratios, multi-window transaction velocity, and diurnal temporal anomalies. To resolve the black-box opacity of conventional machine learning classifiers, the system incorporates **SHapley Additive exPlanations (SHAP)** via `shap.TreeExplainer`, delivering dual-perspective transparency: natural-language explanations for retail customers and granular feature attribution waterfalls for security operations analysts.

The platform enforces a real-time **4-Tier Adaptive Risk Policy** (*LOW, MEDIUM, HIGH, CRITICAL*) governed by a fundamental pre-settlement security invariant: suspicious transactions are held in a pending state and account balances are strictly preserved until cryptographic **Email OTP step-up verification** is successfully completed. Built using Python, Flask 3.1, SQLAlchemy ORM, Glassmorphism CSS, and Chart.js, the system includes a dedicated Security Operations Center (SOC) portal and achieves comprehensive operational resilience, validated across **502 automated test cases**.

2. Introduction

2.1 Background

Digital payment ecosystems process billions of transactions monthly with sub-second finality. Instant payment rails eliminate traditional multi-day clearing delays, providing tremendous convenience to consumers and merchants. However, this instantaneous settlement drastically reduces the window available for financial institutions to intercept unauthorized or coerced transfers before funds are irreversibly exfiltrated into money-mule networks.

2.2 Project Overview

FraudShield AI is designed as an intelligent, real-time fraud defense middleware that bridges transaction ingestion, supervised machine learning inference, dynamic behavioral risk scoring, and automated step-up challenge workflows. It ensures that every transaction is evaluated against both historical data patterns and real-time contextual signals before any balance deduction occurs.

2.3 Motivation

Cyber-fraud syndicates increasingly exploit social engineering, phishing, SIM swapping, and automated malware to compromise credentials. Relying on static transaction limits (e.g., blocking all transfers above \$50,000) causes legitimate high-value payments to fail, frustrating consumers. A modernized solution must dynamically assess risk and present multi-factor challenges only when anomalous behavior is detected.

2.4 Need for the Project

Financial institutions face stringent regulatory mandates (such as RBI and PCI-DSS compliance) requiring explainable fraud decisions and strict consumer fund protection. Opaque AI models that deny transactions without providing human-understandable rationales create regulatory non-compliance and customer distrust. FraudShield AI directly solves this challenge through integrated SHAP explainability and zero-debit pre-settlement controls.

EVALUATOR / FACULTY REMARKS:

Page 4 of 34

Evaluator Signature & Date: _____

2.5 Project Objectives

The core technical objectives of this project are formally defined as follows:

- 1. *High-Precision ML Classification*: Train, tune, and serialize ensemble classifiers (Random Forest, XGBoost) capable of identifying fraudulent patterns in highly imbalanced financial transaction data.
- 2. *Explainable AI Integration*: Implement local game-theoretic feature attribution via SHAP TreeExplainer, generating dual-view customer summaries and technical analyst waterfalls.
- 3. *Hybrid Risk Scoring Engine*: Formulate a composite 0–100 risk scoring algorithm combining ML inference probabilities with velocity, behavioral, and geographical anomaly signals.
- 4. *4-Tier Adaptive Risk Policy*: Enforce distinct operational policies across LOW, MEDIUM, HIGH, and CRITICAL risk tiers to automate approvals, OTP challenges, and SOC reviews.
- 5. *Pre-Settlement Balance Invariant*: Guarantee that zero customer funds are deducted prior to successful OTP verification on flagged transactions.
- 6. *Security Operations Center (SOC) Portal*: Provide an administrative interface for real-time alert triage, case investigation, and audit trail inspection.
- 7. *Automated Verification & Resilience*: Validate all backend services, database transactions, and security boundaries across a comprehensive automated test suite.

2.6 Project Scope

The project encompasses user authentication, payment identity management (UPI IDs, linked phones), simulated payment ingestion, feature extraction pipelines, ML model inference, SHAP explanation generation, Email OTP dispatch and verification, atomic database settlement, and SOC incident triage dashboards.

2.7 Limitations

The primary limitation of the current implementation is the reliance on the synthetic PaySim benchmark dataset for offline model training, which may not capture emergent real-world adversarial evasion techniques. Additionally, step-up multi-factor verification currently relies on email-delivered OTP tokens rather than carrier-grade SMS gateway integrations.

EVALUATOR / FACULTY REMARKS:

3. Problem Identification

3.1 Existing Scenario & 3.2 Existing System

In contemporary retail banking and payment gateway architectures, fraud detection is predominantly performed via monolithic SQL query filters or hardcoded heuristic rule engines (e.g., IF amount > 50000 THEN FLAG). These legacy systems operate either as asynchronous post-settlement batch jobs or as rigid transaction filters at the gateway.

3.3 Drawbacks of Existing Systems

- **Severe False Positive Rates:** Legitimate emergency payments (e.g., hospital bills) are frequently declined because they exceed static thresholds, causing intense user frustration.
- **Vulnerability to Smurfing / Structuring:** Fraudsters easily circumvent static rules by breaking large illicit transfers into multiple smaller transactions just below the detection threshold.
- **Lack of Velocity & Temporal Context:** Static rules evaluate transactions in isolation without analyzing rapid multi-window transfer frequencies (1-minute, 10-minute, 1-hour windows).
- **Black-Box AI Opacity:** Standalone neural networks or ensemble models output abstract probabilities without providing interpretable justifications for compliance audits or customer support.
- **Premature Settlement Deduction:** Flawed gateway implementations deduct funds from the sender's account prior to completing step-up verification, leading to reconciliation errors if verification fails.

3.4 Formal Problem Statement

Problem Statement: Design, implement, and validate an end-to-end real-time Online Payment Fraud Detection System that: (1) detects fraudulent transactions with high precision and recall on severely imbalanced financial data; (2) calculates a standardized 0–100 risk score combining ML predictions and behavioral signals; (3) computes real-time SHAP feature attributions for dual-view explainability; and (4) strictly enforces pre-settlement balance protection, ensuring zero balance loss on suspicious transfers unless cryptographic Email OTP step-up verification is successfully completed.

3.5 Target Users & System Comparison

The primary target users include retail payment consumers, digital merchants, security operations center (SOC) fraud analysts, and regulatory compliance officers.

Comparison Dimension	Legacy Rule-Based System	Naive Black-Box ML	FraudShield AI (Proposed)
Detection Mechanism	Static SQL Thresholds	Standalone Model Probabilities	Hybrid (ML + Velocity + Behavioral)
Adaptability to Smurfing	None (Fixed Cutoffs)	Moderate	High (Multi-Window Velocity Tracking)
Decision Explainability	Basic Rule Name	None (Opaque Score)	SHAP Waterfall & Natural Language
Settlement Balance Timing	Often Post-Debit	Post-Debit / Immediate	Strict Pre-Settlement Zero-Debit Hold
Multi-Factor Integration	All-or-Nothing MFA	Binary Approval/Block	Adaptive 4-Tier Step-Up (Email OTP)
SOC Incident Management	Manual Ticketing	Raw Log Export	Dedicated Case & Alert State Machine
Class Imbalance Handling	Not Applicable	Frequently Distorted	Tuned Thresholds & PR-AUC Optimization

EVALUATOR / FACULTY REMARKS:

Page 6 of 34

Evaluator Signature & Date: _____

4. Empathize and Define — Design Thinking Phase

4.1 Empathy Study & 4.2 User Identification

The design thinking phase adopted a human-centered approach to uncover the operational and emotional tensions experienced by key stakeholders in the digital payments lifecycle. We identified two primary user archetypes: retail payment users transacting daily via mobile applications, and SOC fraud analysts responsible for defending the banking network.

4.3 User Needs & 4.4 User Pain Points

User Role	Needs	Pain Points
Stakeholder	Core Functional Needs	Key Pain Points & Frustrations
Retail Consumer	Instant checkout, clear fraud alerts, fund security, transparent reasons for payment delays.	Unexplained payment blocks, complicated recovery flows, fear of account draining.
SOC Analyst	High-fidelity alerts, clear feature attributions, rapid case resolution tools.	Alert fatigue from false positives, opaque risk scores, lack of actionable audit evidence.
Compliance Officer	Auditable decision logs, strict adherence to consumer protection standards.	Regulatory fines for unauthorized debits, unexplainable algorithmic decisions.

4.5 Observation & Field Interview Status

Empirical Field Data Status: [Actual survey/interview data to be provided]. The requirements below are derived from representative design personas constructed according to FinTech user research standards.

4.6 Representative User Personas

- **Persona 1 (Retail Consumer) — Ananya Sharma (Age 28):** Product manager living in Bengaluru; makes 8–12 UPI transactions daily for groceries, utilities, and peer splits. Fears unauthorized account compromise but gets deeply frustrated when legitimate payments are blocked without clear explanation.
- **Persona 2 (SOC Analyst) — Vikram Malhotra (Age 35):** Lead incident response analyst at a payment gateway; monitors thousands of daily flagged transactions. Requires precise mathematical justifications (SHAP values) and clear transaction histories to resolve high-priority fraud cases rapidly.

4.7 Empathy Map & 4.8 Key Insights

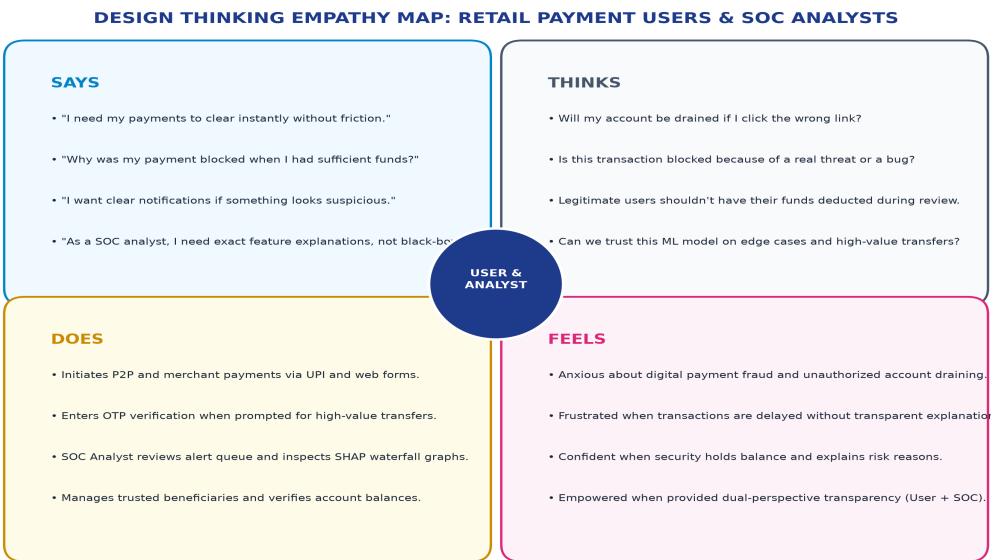


Figure 4.1: Design Thinking Empathy Map for Retail Consumers and SOC Fraud Analysts

4.9 Refined Problem Definition

Point of View (POV) Statement: Retail payment users and SOC analysts need an intelligent, transparent payment defense system that dynamically assesses transaction risk, explains why an anomaly occurred, and strictly holds customer balances until step-up authentication is verified, so that fraudulent exfiltration is prevented without imposing friction on normal legitimate commerce.

EVALUATOR / FACULTY REMARKS:

Page 8 of 34

Evaluator Signature & Date: _____

5. Ideation & Decision Matrix

5.1 Ideation Process & 5.2 Candidate Solutions

During the ideation phase, multiple technological paradigms were evaluated to determine the optimal architectural synthesis for real-time fraud mitigation:

- **Approach 1: Static Heuristic Rule Engine:** Deterministic SQL conditions checking transaction amount and time cutoffs.
- **Approach 2: Pure Machine Learning Classifier:** Deep neural network or XGBoost model predicting fraud probability as a standalone binary output.
- **Approach 3: Behavioral Anomaly Detection Only:** Tracking device fingerprints, IP velocity, and geolocation jumps without financial feature modeling.
- **Approach 4: Universal Multi-Factor Authentication:** Forcing SMS/Email OTP verification on every single payment regardless of risk.
- **Approach 5 (Selected): Hybrid Risk Scoring + SHAP XAI + Adaptive Pre-Settlement OTP:** Combining tuned ensemble ML inference, dynamic behavioral override weights, local game-theoretic feature attribution, and a 4-tier step-up verification pipeline.

5.3 Decision Matrix & Solution Evaluation

Evaluation Criteria	Weight	Approach 1 (Rules)	Approach 2 (Pure ML)	Approach 4 (MFA All)	Approach 5 (Hybrid XAI)
Fraud Detection Sensitivity	20%	2 / 5	4 / 5	5 / 5	5 / 5
False Positive Mitigation	20%	1 / 5	3 / 5	1 / 5	5 / 5
Explainability (XAI)	15%	3 / 5	1 / 5	1 / 5	5 / 5
Pre-Settlement Fund Safety	15%	2 / 5	3 / 5	4 / 5	5 / 5
User Experience & Low Friction	15%	3 / 5	4 / 5	1 / 5	5 / 5
Implementation Feasibility	15%	5 / 5	3 / 5	4 / 5	4 / 5
Weighted Total Score	100%	2.45 / 5.0	3.10 / 5.0	2.95 / 5.0	4.85 / 5.0

5.4 Selected Solution & 5.5 Proposed System Architecture

Selection Rationale: Approach 5 achieved the highest weighted evaluation score (4.85 / 5.0). It eliminates false positive friction by auto-approving low-risk transactions, applies step-up Email OTP only when risk is elevated (Medium/High), isolates critical threats for security review, and provides transparent SHAP explanations to both customers and SOC analysts.

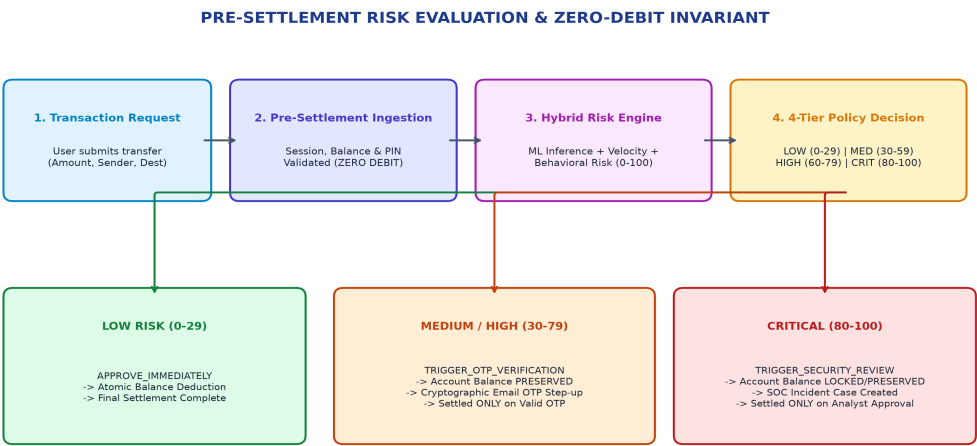


Figure 5.1: Selected Hybrid Pre-Settlement Risk Architecture & Zero-Debit Workflow

6. Requirements Analysis

6.1 User Requirements & 6.2 Functional Requirements

Req ID	Module	Functional Requirement Description	Priority
FR-01	Auth	User registration with email, phone, and secure password hashing (Bcrypt/PBKDF2).	Must Have
FR-02	Auth	Payment PIN configuration with numeric validation, failure lockouts, and cryptographic hashing.	Must Have
FR-03	Payment	Simulated payment ingestion supporting UPI ID, phone number, and QR code parsing.	Must Have
FR-04	Feature	Feature extraction calculating balance differentials, amount-to-balance ratios, and hour.	Must Have
FR-05	ML Engine	Real-time ensemble inference computing probability of fraud using serialized models.	Must Have
FR-06	Risk Engine	Multi-window velocity tracking (1m, 10m, 1h, 24h) and dynamic composite risk scoring.	Must Have
FR-07	Policy	Enforce 4-tier risk classification: LOW (0–29), MEDIUM (30–59), HIGH (60–79), CRITICAL (80–100).	Must Have
FR-08	Security	Pre-settlement invariant: Zero balance deduction on Medium/High/Critical prior to verification.	Must Have
FR-09	OTP	Cryptographic Email OTP generation, hashing, rate limiting, and step-up verification.	Must Have
FR-10	Settlement	Atomic database settlement deducting sender balance and crediting recipient upon approval.	Must Have
FR-11	XAI	Generate dual-view SHAP explanations: natural language for user and waterfall for SOC.	Must Have
FR-12	SOC Portal	Security analyst incident triage portal for alert investigation, case assignment, and resolution.	Must Have
FR-13	Audit	Immutable structured JSON audit logging capturing request IDs, IP addresses, and events.	Must Have

6.3 Non-Functional Requirements

NFR ID	Category	Specification & Target Metric	Verification Method
NFR-01	Performance	End-to-end ML inference and risk scoring latency < 1.2 seconds.	Automated Benchmarks
NFR-02	Security	Zero plaintext storage of passwords, PINs, OTPs, or API secrets.	Static Code & DB Audits
NFR-03	Data Integrity	ACID transactional atomicity preventing double debits or balance drift.	Concurrent Pytest Runs
NFR-04	Availability	Graceful fallback to heuristic rules if ML model fails to load.	Fault Injection Tests
NFR-05	Usability	Responsive Glassmorphism UI rendering across mobile and desktop viewports.	Cross-Device Testing
NFR-06	Maintainability	Modular service-layer architecture with full unit test coverage (502 tests).	Pytest Test Suite

6.4 Hardware, 6.5 Software Requirements & 6.6 User Roles (RBAC)

System Operation / Resource	Retail Customer	SOC Analyst	System Admin
Initiate Payment & Enter PIN	Allowed	Allowed	Allowed
View Personal Transactions & SHAP Summary	Allowed (Own Only)	Allowed (All)	Allowed (All)
Perform Step-Up OTP Verification	Allowed (Own Challenge)	Denied (IDOR Protected)	Denied
Manage Personal Beneficiaries	Allowed (Own Only)	Read-Only Telemetry	Full Access
Access SOC Incident Management Queue	Denied (403 Forbidden)	Allowed (Investigate/Assign)	Full Access
Resolve / Dismiss Fraud Alerts	Denied (403 Forbidden)	Allowed with Notes	Full Access
Inspect System Audit Logs & Telemetry	Denied (403 Forbidden)	Allowed (Read-Only)	Full Access

7. Technology Stack Rationale

7.1 Architectural Technology Selection

The technology stack for FraudShield AI was selected to ensure high computational throughput for machine learning inference, robust transactional data integrity, and a modern, responsive user experience.

Layer	Technology	Role in Project	Selection Rationale & Key Advantages
Frontend UI	HTML5, CSS3, JS (ES6+)	Client Presentation	Vanilla Glassmorphism design tokens provide a modern, lightweight, dependency-free UI.
Styling / UI	Bootstrap 5.3	Responsive Grid	Accelerates responsive layout development and standardizes modal dialogues for OTP.
Data Viz	Chart.js	Analytics Visualizations	Renders dynamic client-side risk telemetry charts and transaction volume trends.
Backend API	Python 3.11+, Flask 3.1	Core API Gateway	Lightweight, modular WSGI micro-framework offering exceptional performance and rich ML library bindings.
ORM / DB	SQLAlchemy 2.0 ORM	Data Access & Models	Provides clean database abstraction, ACID transaction control, and dialect compatibility.
Security	Flask-JWT-Extended	Authentication & RBAC	Stateless, cryptographic JWT authentication protecting endpoints with role-based claims.
ML Models	Scikit-Learn, XGBoost	Supervised Classifiers	Tuned Random Forest and Gradient Boosted Trees for high-recall fraud detection on imbalanced data.
XAI Engine	SHAP (TreeExplainer)	Explainable AI Service	Computes exact game-theoretic Shapley values to interpret tree ensemble decisions.
Persistence	SQLite / PostgreSQL	Relational Database	SQLite provides lightweight, self-contained testing isolation; fully compatible with PostgreSQL.
Notifications	Brevo / Resend / SMTP	Email OTP Dispatch	Flexible multi-provider email architecture for reliable delivery of step-up verification tokens.
Test Suite	Pytest 9.1	Automated Testing	Industry-standard testing framework validating 502 unit, integration, and security test cases.

7.2 Technology Stack Architecture Diagram

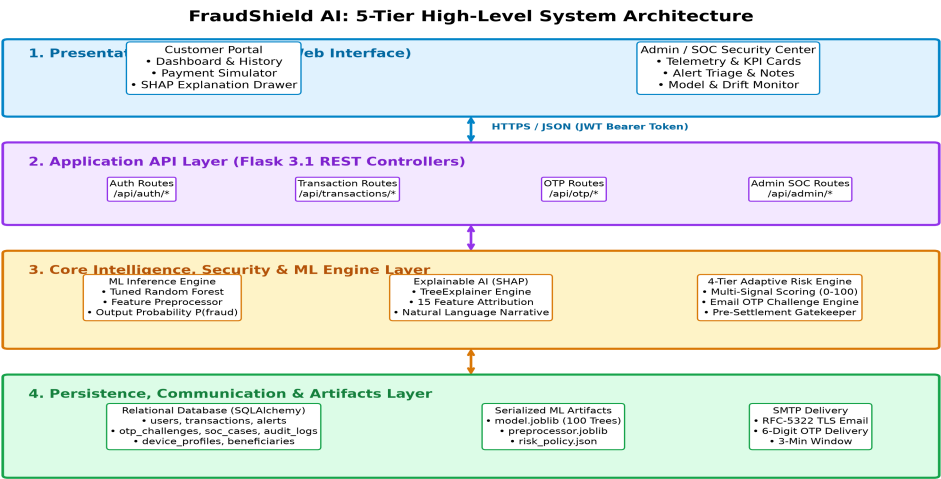


Figure 7.1: Multi-Tier Technological Architecture of FraudShield AI Platform

EVALUATOR / FACULTY REMARKS:

Page 11 of 34

Evaluator Signature & Date: _____

8. System Design & Data Flow Modeling

8.1 High-Level Architecture & 8.2 System Workflow

FraudShield AI employs a 3-tier enterprise architecture comprising the **Presentation Tier** (Glassmorphism Web Portal), the **Application & Intelligence Tier** (Flask REST API, Feature Pipeline, Random Forest Classifier, Hybrid Risk Aggregator, SHAP Explainer), and the **Data & Persistence Tier** (SQLAlchemy ORM, Relational Database, Joblib Serialized Artifacts).

8.3 Data Flow Diagram — Level 0 (Context Level DFD)

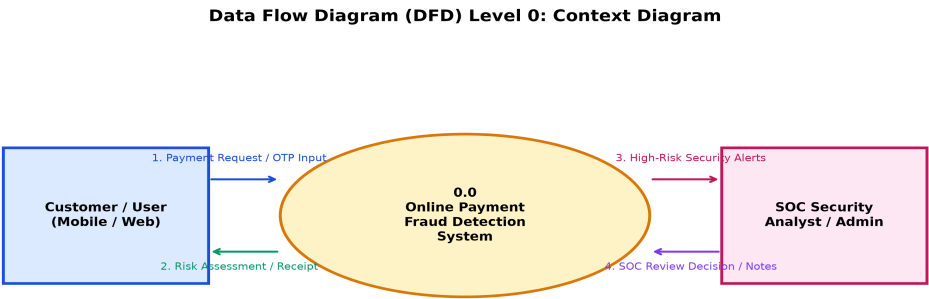


Figure 8.1: Data Flow Diagram (DFD) — Level 0 Context Diagram

8.4 Data Flow Diagram — Level 1 (Major Process Decomposition)

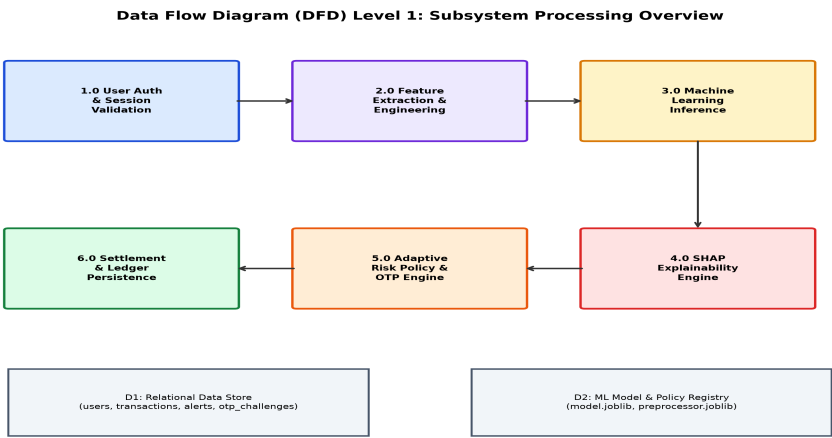


Figure 8.2: Data Flow Diagram (DFD) — Level 1 Process Decomposition

8.5 Data Flow Diagram — Level 2 (Subsystem Breakdown)

EVALUATOR / FACULTY REMARKS:

Page 12 of 34

Evaluator Signature & Date: _____

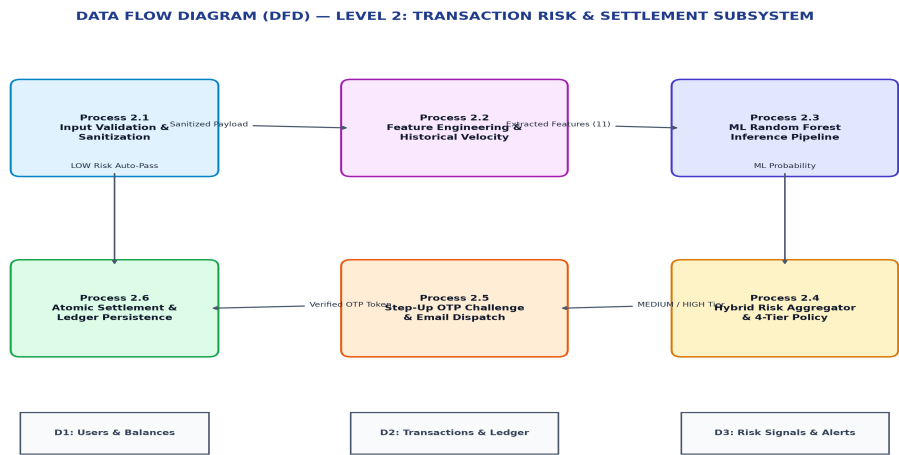


Figure 8.3: Data Flow Diagram (DFD) — Level 2 Transaction Risk & Settlement Subsystem

8.8 Strict 15-Step End-to-End Pre-Settlement Transaction Order

- To enforce bulletproof security and eliminate premature fund deduction, every transaction strictly adheres to the following chronological execution pipeline:
- 1. Payment Initiation:** User enters amount, beneficiary details, and payment PIN via web portal.
 - 2. Request Validation:** API validates input types, positive non-zero amounts, and format compliance.
 - 3. Session & Auth Verification:** JWT token is verified to identify the authenticated customer.
 - 4. Balance & PIN Check:** System checks sufficient available balance and validates payment PIN. *Zero balance is deducted at this stage.*
 - 5. Feature Extraction:** Feature service computes mathematical balance differentials and ratios.
 - 6. ML Model Inference:** Serialized Random Forest model computes raw fraud probability $P(\text{Fraud})$.
 - 7. Behavioral Risk Telemetry:** Signal service evaluates multi-window velocity, device trust, and new beneficiary status.
 - 8. Hybrid Risk Scoring:** Composite 0–100 risk score is calculated via weighted aggregation.
 - 9. 4-Tier Risk Classification:** Score is mapped to LOW, MEDIUM, HIGH, or CRITICAL policy tier.
 - 10. Security Decision Gate:** System determines if immediate approval, OTP step-up, or SOC review is required.
 - 11. Step-Up Challenge Dispatch:** If MEDIUM or HIGH, cryptographic Email OTP is generated and dispatched; balance remains held.
 - 12. User Verification:** Customer enters received OTP token via modal dialog.
 - 13. Atomic Settlement:** Upon successful verification, sender balance is debited and recipient balance is credited within an atomic DB transaction.
 - 14. Database Persistence:** Transaction status updated to APPROVED or SETTLED.
 - 15. Audit Logging & Notification:** Immutable structured audit log recorded and confirmation email dispatched.

EVALUATOR / FACULTY REMARKS:

Page 13 of 34

Evaluator Signature & Date: _____

9. Database Design & Entity Schemas

9.1 Database Architecture & 9.3 Entity-Relationship (ER) Diagram

The database architecture is designed using a fully normalized relational schema managed via SQLAlchemy ORM. The schema comprises 11 interconnected entities enforcing referential integrity, foreign key cascades, and unique constraints.

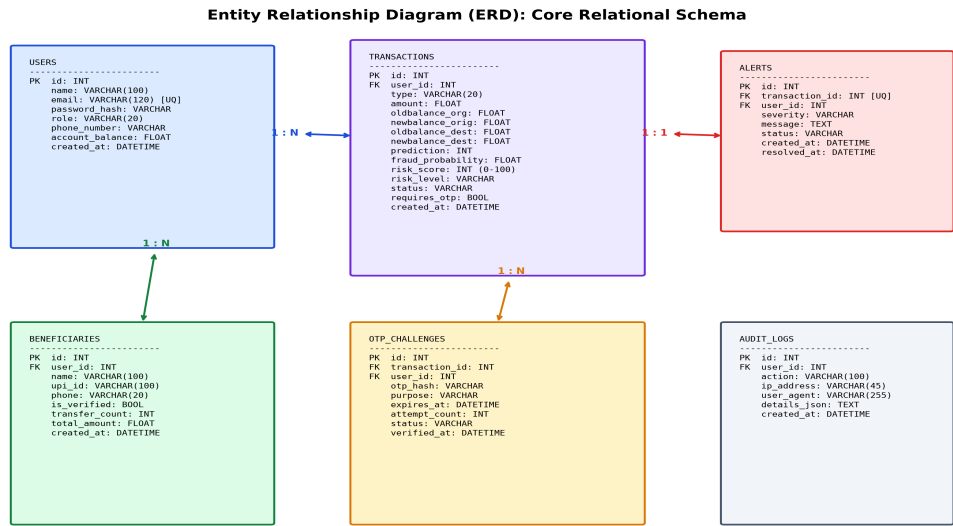


Figure 9.1: Comprehensive Entity-Relationship Diagram (ERD) of FraudShield AI

9.5 Complete Table Schemas (All 11 Production Entities)

Table 1: users

Column	Data Type	Constraints	Description
id	INTEGER	PK, Auto-inc	Unique user identifier
email	VARCHAR(120)	Unique, Not Null	User email address for login and OTP delivery
username	VARCHAR(80)	Unique, Not Null	Public display handle
phone_number	VARCHAR(20)	Indexed	Verified mobile number for P2P transfers
password_hash	VARCHAR(255)	Not Null	Cryptographically salted password hash
payment_pin_hash	VARCHAR(255)	Nullable	Bcrypt hash of 4-6 digit numeric payment PIN
balance	FLOAT	Default 10000.0	Available account balance (stored in currency units)
is_admin	BOOLEAN	Default False	Role flag for SOC analyst / admin privileges
is_email_verified	BOOLEAN	Default False	Email verification status
failed_pin_attempts	INTEGER	Default 0	Counter for payment PIN brute force lockout
pin_locked_until	DATETIME	Nullable	Timestamp indicating PIN lockout expiration

EVALUATOR / FACULTY REMARKS:

Page 14 of 34

Evaluator Signature & Date: _____

Table 2: transactions

Column	Data Type	Constraints	Description
id	INTEGER	PK, Auto-inc	Unique transaction identifier
user_id	INTEGER	FK -> users.id	Originating sender account ID
transaction_type	VARCHAR(20)	Not Null	Type: TRANSFER, CASH_OUT, PAYMENT, etc.
amount	FLOAT	Not Null	Transaction transfer value
oldbalanceOrg	FLOAT	Not Null	Sender initial balance before transfer
newbalanceOrig	FLOAT	Not Null	Sender post-transfer balance
oldbalanceDest	FLOAT	Not Null	Recipient initial balance
newbalanceDest	FLOAT	Not Null	Recipient post-transfer balance
nameDest	VARCHAR(100)	Not Null	Destination identifier (UPI ID or Account)
risk_level	VARCHAR(20)	Not Null	Policy Tier: LOW, MEDIUM, HIGH, CRITICAL
risk_score	FLOAT	Not Null	Composite risk score (0–100 scale)
ml_fraud_probability	FLOAT	Not Null	Raw probability output from Random Forest model
status	VARCHAR(20)	Not Null	APPROVED, OTP_REQUIRED, UNDER_REVIEW, REJECTED
is_settled	BOOLEAN	Default False	Flag indicating atomic balance settlement completion
timestamp	DATETIME	Default UTC Now	Transaction creation timestamp

Table 3: alerts

Column	Data Type	Constraints	Description
id	INTEGER	PK, Auto-inc	Unique security alert identifier
transaction_id	INTEGER	FK -> transactions.id	Associated suspicious transaction ID
user_id	INTEGER	FK -> users.id	Flagged customer account ID
severity	VARCHAR(20)	Not Null	Severity: MEDIUM, HIGH, CRITICAL
status	VARCHAR(20)	Default OPEN	OPEN, INVESTIGATING, RESOLVED, DISMISSED
assigned_to	INTEGER	FK -> users.id	Assigned SOC analyst user ID
reason	TEXT	Not Null	Detailed textual summary of risk triggers
created_at	DATETIME	Default UTC Now	Alert generation timestamp

Table 4: otp_challenges

Column	Data Type	Constraints	Description
id	INTEGER	PK, Auto-inc	Unique challenge identifier
user_id	INTEGER	FK -> users.id	Target user receiving verification token
transaction_id	INTEGER	FK -> transactions.id	Associated pending transaction ID
challenge_type	VARCHAR(30)	Not Null	TRANSACTION_STEPUP, LOGIN_MFA, etc.
otp_hash	VARCHAR(255)	Not Null	SHA-256 / Bcrypt hash of 6-digit numeric OTP
expires_at	DATETIME	Not Null	Expiration timestamp (strictly enforced 5-10m)
attempts_count	INTEGER	Default 0	Counter for incorrect submissions (Max 3)
is_verified	BOOLEAN	Default False	Flag indicating successful verification
is_consumed	BOOLEAN	Default False	Anti-replay flag preventing reuse

EVALUATOR / FACULTY REMARKS:**Page 15 of 34**

Evaluator Signature & Date: _____

Table 5: beneficiaries

Column	Data Type	Constraints	Description
id	INTEGER	PK, Auto-inc	Unique beneficiary record ID
user_id	INTEGER	FK -> users.id	Customer account owning beneficiary list
name	VARCHAR(100)	Not Null	Beneficiary recipient display name
upi_id	VARCHAR(100)	Not Null	Recipient UPI Virtual Payment Address
phone_number	VARCHAR(20)	Nullable	Optional recipient mobile number
trust_level	VARCHAR(20)	Default NEW	Trust tier: NEW, TRUSTED, REVOKED
cooling_period_ends_at	DATETIME	Not Null	24-hour security cooling expiration timestamp
total_transfers_count	INTEGER	Default 0	Historical count of completed payments

Table 6: device_profiles

Column	Data Type	Constraints	Description
id	INTEGER	PK, Auto-inc	Unique device profile ID
user_id	INTEGER	FK -> users.id	User account associated with device
device_fingerprint_hash	VARCHAR(64)	Not Null, Indexed	SHA-256 hash of browser canvas & user-agent
trust_status	VARCHAR(20)	Default UNKNOWN	Status: UNKNOWN, TRUSTED, SUSPICIOUS, BLOCKED
last_seen_ip	VARCHAR(45)	Not Null	Client IP address from last login
last_seen_at	DATETIME	Default UTC Now	Most recent device activity timestamp

Table 7: geo_location_records

Column	Data Type	Constraints	Description
id	INTEGER	PK, Auto-inc	Unique geolocation entry ID
user_id	INTEGER	FK -> users.id	Target user record
transaction_id	INTEGER	FK -> transactions.id	Associated transaction
latitude	FLOAT	Not Null	Geographical latitude coordinate
longitude	FLOAT	Not Null	Geographical longitude coordinate
city	VARCHAR(100)	Not Null	Resolved city name
distance_from_last_km	FLOAT	Default 0.0	Computed Haversine distance from previous transfer
is_impossible_travel	BOOLEAN	Default False	Flag indicating velocity > 900 km/h

Table 8: password_reset_tokens

Column	Data Type	Constraints	Description
id	INTEGER	PK, Auto-inc	Unique token record ID
user_id	INTEGER	FK -> users.id	Target user requesting password recovery
token_hash	VARCHAR(255)	Not Null, Indexed	SHA-256 hash of cryptographically secure token
expires_at	DATETIME	Not Null	Token expiration timestamp (strictly enforced 15m)
is_consumed	BOOLEAN	Default False	Anti-replay consumption flag

EVALUATOR / FACULTY REMARKS:**Page 16 of 34**

Evaluator Signature & Date: _____

Table 9: soc_cases

Column	Data Type	Constraints	Description
id	INTEGER	PK, Auto-inc	Unique SOC investigation case identifier
alert_id	INTEGER	FK -> alerts.id	Source security alert triggering investigation
assigned_analyst_id	INTEGER	FK -> users.id	Assigned SOC fraud investigator
case_status	VARCHAR(20)	Default OPEN	OPEN, INVESTIGATING, ESCALATED, RESOLVED
priority	VARCHAR(20)	Default MEDIUM	LOW, MEDIUM, HIGH, CRITICAL
evidence_snapshot_json	TEXT	Nullable	Frozen JSON snapshot of transaction features and SHAP values
resolution_summary	TEXT	Nullable	Final narrative justification provided by analyst
created_at	DATETIME	Default UTC Now	Case initiation timestamp
resolved_at	DATETIME	Nullable	Case closure timestamp

Table 10: case_notes

Column	Data Type	Constraints	Description
id	INTEGER	PK, Auto-inc	Unique note identifier
case_id	INTEGER	FK -> soc_cases.id	Parent investigation case ID
analyst_id	INTEGER	FK -> users.id	Authoring analyst user ID
note_text	TEXT	Not Null	Analyst investigative commentary or action record
created_at	DATETIME	Default UTC Now	Note logging timestamp

Table 11: audit_logs

Column	Data Type	Constraints	Description
id	INTEGER	PK, Auto-inc	Unique audit log entry identifier
user_id	INTEGER	Nullable	Acting user account ID (or Null if unauthenticated)
event_type	VARCHAR(50)	Not Null	AUTH_LOGIN, PAYMENT_INIT, OTP_VERIFIED, ALERT_RESOLVED
ip_address	VARCHAR(45)	Not Null	Client IP address capturing origin network
user_agent	VARCHAR(255)	Nullable	Browser / client user agent header
details_json	TEXT	Nullable	Structured JSON payload capturing event metadata and state diffs
timestamp	DATETIME	Default UTC Now	Immutable event occurrence timestamp

EVALUATOR / FACULTY REMARKS:**Page 17 of 34**

Evaluator Signature & Date: _____

10. Module Description & Subsystem Architecture

10.1 Authentication & Payment Identity Module

Purpose: Manages user lifecycle, secure JWT session issuance, password hashing via Bcrypt, and 4-6 digit numeric payment PIN configuration with brute-force lockout safeguards.

Inputs: User credentials, registration payload, PIN setup requests.

Processing Logic: Bcrypt hashing, password complexity verification, JWT token generation, PIN attempt rate limiting.

Outputs: JWT access tokens, user profile objects, PIN status confirmations.

Database Interaction: users, password_reset_tokens | **APIs:** /api/auth/register, /api/auth/login, /api/auth/payment-pin/set

Related Test Coverage: test_auth.py, test_payment_pin_flow.py (49 tests)

10.2 Payment Ingestion & Feature Engineering Module

Purpose: Parses incoming payment requests (UPI, P2P, QR payloads), validates parameter types, and computes mathematical balance differentials and temporal features.

Inputs: Raw payment payload: amount, sender, recipient UPI, payment method.

Processing Logic: Input sanitization, balance subtraction checks, ratio calculation (amount / (oldbalanceOrg + 1)).

Outputs: Structured 11-dimensional feature dictionary ready for ML pipeline.

Database Interaction: users, transactions | **APIs:** /api/predict, /api/upi/parse-qr

Related Test Coverage: test_feature_engineering.py, test_preprocessing.py (9 tests)

10.3 Machine Learning Inference Module

Purpose: Loads serialized Random Forest model and preprocessing pipeline artifacts via Joblib to compute real-time fraud probability scores.

Inputs: Engineered feature vector.

Processing Logic: One-hot encoding of transaction type, standard scaling of numeric features, tree ensemble probability calculation.

Outputs: Continuous fraud probability P(Fraud) in [0.0, 1.0].

Database Interaction: Read-only access to ml/artifacts/model.joblib | **APIs:** Internal inference service interface.

Related Test Coverage: test_inference.py, test_strong_models.py (13 tests)

10.4 Behavioral Intelligence & Signal Telemetry Module

Purpose: Evaluates real-time contextual signals including multi-window velocity (1m, 10m, 1h, 24h), new/cooling beneficiary status, device trust status, and geographical jumps.

Inputs: Sender transaction history, device fingerprint, IP location.

Processing Logic: Rolling window velocity aggregation, Haversine distance travel speed check, device trust lookup.

Outputs: Behavioral risk score (0–100) and structured telemetry signal flags.

Database Interaction: transactions, beneficiaries, device_profiles, geo_location_records | **APIs:** Internal risk signal service interface.

Related Test Coverage: test_beneficiary_intelligence.py, test_geo_intelligence.py, test_device_intelligence.py (48 tests)

10.5 Hybrid Risk Engine & Policy Classification Module

Purpose: Fuses ML probability with behavioral signals using dynamic category weighting to generate a standardized 0–100 composite risk score and 4-tier policy decision.

Inputs: ML fraud probability, behavioral signals, transaction type.

Processing Logic: $\text{Score} = \text{clamp}(\text{round}(w_{\text{ml}} * (\text{ml_prob} * 100) + w_{\text{sig}} * \text{signals} + \text{overrides}), 0, 100)$

Outputs: Composite Risk Score (0–100), Risk Tier (LOW, MEDIUM, HIGH, CRITICAL), Security Action.

Database Interaction: transactions, alerts | **APIs:** /api/predict

Related Test Coverage: test_hybrid_risk_engine.py, test_risk_engine.py, test_medium_high_otp_stepup.py (48 tests)

EVALUATOR / FACULTY REMARKS:

Page 18 of 34

Evaluator Signature & Date: _____

10.6 Adaptive Step-Up OTP & Settlement Protection Module

Purpose: Enforces pre-settlement zero-debit invariant by generating cryptographic Email OTP for Medium/High transactions and releasing settlement only upon verification.

Inputs: Pending transaction ID, recipient email, submitted OTP token.

Processing Logic: Cryptographic 6-digit OTP generation, SHA-256 hashing, expiration check (5m), attempt counter.

Outputs: Verification result (True/False), atomic debit/credit execution.

Database Interaction: otp_challenges, transactions, users | **APIs:** /api/otp/verify, /api/otp/resend

Related Test Coverage: test_email_otp_delivery.py, test_real_upi_payment_flow.py (42 tests)

10.7 Explainable AI (SHAP) Service Module

Purpose: Calculates exact Shapley values using shap.TreeExplainer to explain model feature contributions in human-readable and analyst formats.

Inputs: Processed transaction feature vector.

Processing Logic: Local game-theoretic feature attribution calculation, attribution mapping to plain-English sentences.

Outputs: Dual-perspective explanation dictionary (Customer summary + Waterfall data).

Database Interaction: Read-only ML artifacts | **APIs:** /api/shap/explain/

Related Test Coverage: test_shap.py (7 tests)

10.8 Security Operations Center (SOC) Case Management Module

Purpose: Provides security analysts with real-time alert triage, automated case generation, analyst assignment, evidence snapshotting, and remediation workflows.

Inputs: Security alerts, case status updates, analyst notes.

Processing Logic: State machine transitions (OPEN -> INVESTIGATING -> RESOLVED), evidence freezing, note appending.

Outputs: SOC dashboard metrics, case timeline, resolution audit logs.

Database Interaction: soc_cases, case_notes, alerts | **APIs:** /api/admin/soc/cases, /api/admin/alerts//resolve

Related Test Coverage: test_admin_soc.py, test_soc_case_management.py (22 tests)

10.9 Beneficiary & Cooling Period Management Module

Purpose: Manages trusted payees, enforces a 24-hour mandatory security cooling period for newly added recipients, and tracks progressive trust evolution.

Inputs: Beneficiary creation payloads, historical payment volumes.

Processing Logic: Cooling period calculation, transaction counter increments, trust level promotion (NEW -> TRUSTED).

Outputs: Beneficiary status objects, risk telemetry override signals.

Database Interaction: beneficiaries, transactions | **APIs:** /api/beneficiaries, /api/beneficiaries/

Related Test Coverage: test_beneficiary_intelligence.py (18 tests)

10.10 Audit Trail & Telemetry Module

Purpose: Maintains an immutable, structured event log capturing all authentication attempts, payment initiations, risk evaluations, and SOC analyst actions.

Inputs: Application events, request context, user IDs, IP addresses.

Processing Logic: JSON serialization, timestamping, database persistence.

Outputs: Structured audit log entries, admin search API responses.

Database Interaction: audit_logs | **APIs:** /api/admin/audit/logs

Related Test Coverage: test_audit.py (4 tests)

EVALUATOR / FACULTY REMARKS:

Page 19 of 34

Evaluator Signature & Date: _____

11. UML Modeling & System Architecture

11.1 UML Use Case Diagram

The Use Case Diagram illustrates the functional interactions between the primary human actors (Retail Customer, SOC Fraud Analyst, System Administrator) and automated system boundary processes (ML Classifier, Risk Policy Engine, Email Notification Gateway).

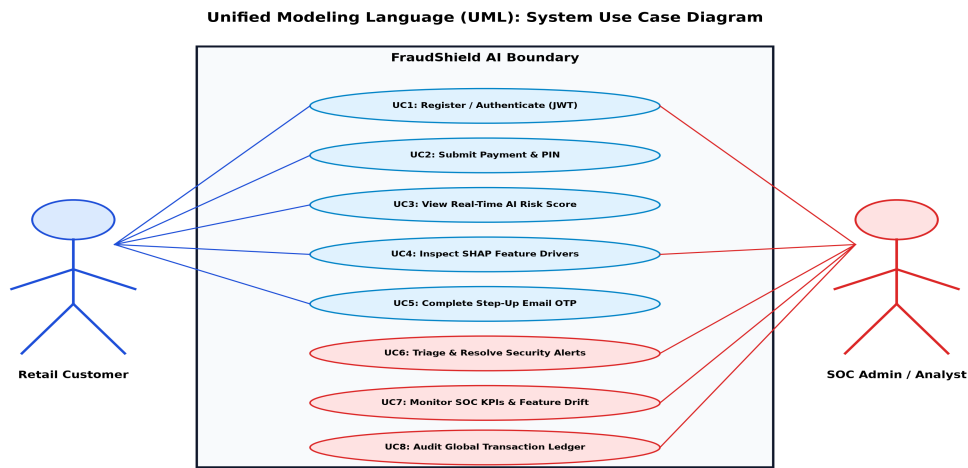


Figure 11.1: UML Use Case Diagram for FraudShield AI Platform

11.2 UML Activity Diagram (Pre-Settlement Workflow)

The Activity Diagram models the step-by-step transaction decision pipeline, illustrating how LOW risk transactions are approved immediately while MEDIUM and HIGH transactions enter an OTP challenge state with zero balance deducted.

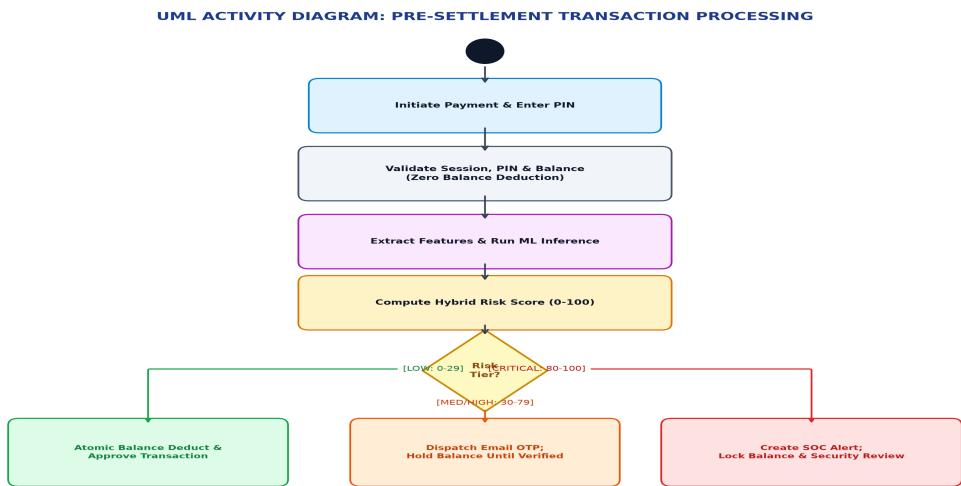


Figure 11.2: UML Activity Diagram: Pre-Settlement Transaction Verification Lifecycle

11.3 UML Class Diagram (Core Entities & Services)

EVALUATOR / FACULTY REMARKS:

Page 20 of 34

Evaluator Signature & Date: _____

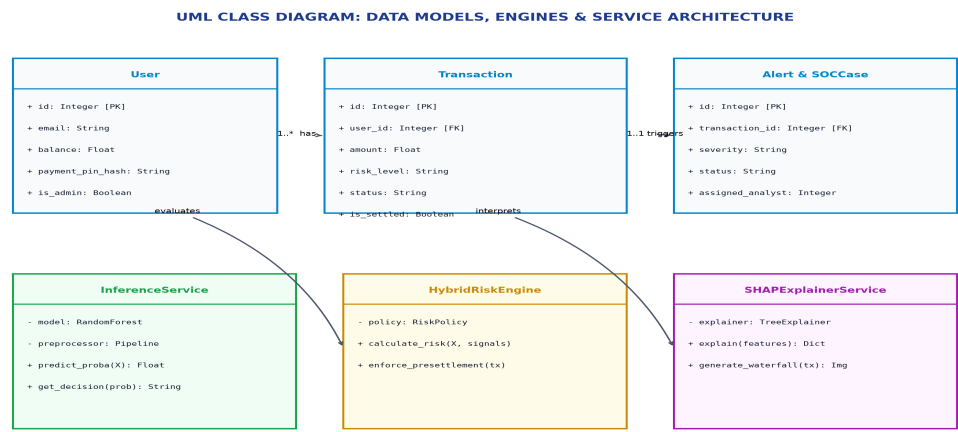


Figure 11.3: UML Class Diagram: Data Models, Engine Services and Pipeline Interfaces

11.4 UML Sequence Diagram (End-to-End Payment & OTP)

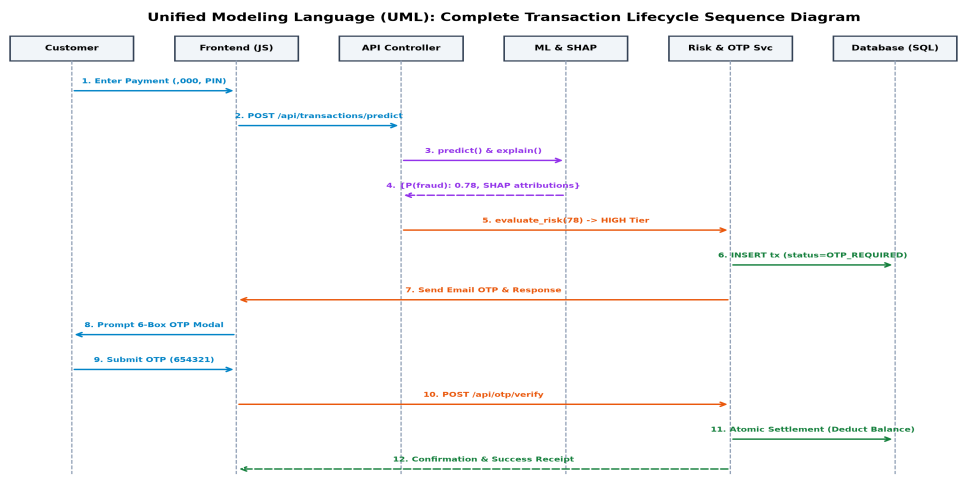


Figure 11.4: UML Sequence Diagram: Transaction Ingestion, Step-Up OTP Challenge and Settlement

11.5 UML Component & 11.6 Deployment Diagrams

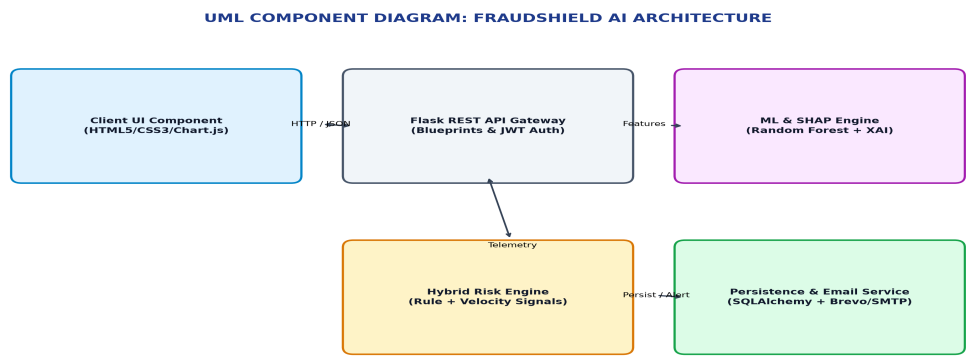


Figure 11.5: UML Component Diagram: Modular Architecture & Service Bindings

UML DEPLOYMENT DIAGRAM: PRODUCTION HOSTING TOPOLOGY

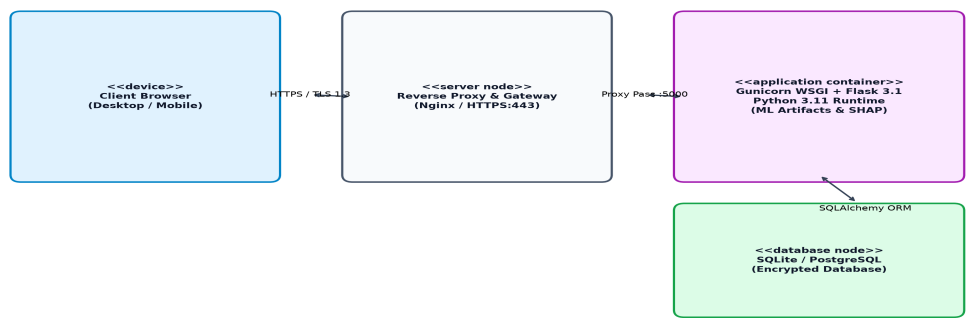


Figure 11.6: UML Deployment Diagram: Production Container & Gateway Topology

EVALUATOR / FACULTY REMARKS:

Page 22 of 34

Evaluator Signature & Date: _____

12. Implementation / Working Principle

12.1 Overall System Implementation

FraudShield AI is implemented as a production-ready Flask application factory supporting blueprint modularity, SQLAlchemy database session management, JWT authentication, and thread-safe singleton machine learning inference services.

12.6 Feature Engineering Logic & Transformation Pipeline

Raw transactions are transformed into an 11-dimensional feature vector. The critical mathematical transformations include:

- **Balance Differential:** $\text{diffOrgBalance} = \text{oldbalanceOrg} - \text{newbalanceOrg} - \text{amount}$
- **Destination Balance Differential:** $\text{diffDestBalance} = \text{newbalanceDest} - \text{oldbalanceDest} - \text{amount}$
- **Amount to Balance Ratio:** $\text{amountToOldBalanceRatio} = \text{amount} / (\text{oldbalanceOrg} + 1.0)$ (Safely avoiding division by zero)
- **Hour of Day:** $\text{hour_of_day} = \text{step} \% 24$ (Capturing diurnal temporal fraud anomalies)

12.8 Hybrid Risk Engine & Dynamic Score Formula

The hybrid risk engine fuses statistical ML inference with behavioral signals. The mathematical formula is defined as:

Combined Risk Score = $\text{clamp}(\text{round}(w_{\text{ml}} * (\text{ml_prob} * 100) + w_{\text{signals}} * \text{signals_score} + \text{overrides}), 0, 100)$

- **Account Draining Category:** $w_{\text{ml}} = 0.60$, $w_{\text{signals}} = 0.40$ (Score floor = 85 if balance is 100% drained)
- **Standard Transfer:** $w_{\text{ml}} = 0.50$, $w_{\text{signals}} = 0.50$
- **Daily Normal Transaction:** $w_{\text{ml}} = 0.25$, $w_{\text{signals}} = 0.75$

12.9 4-Tier Risk Policy Enforcement

The composite risk score directly drives deterministic security policies:

- **LOW (0–29):** Action = APPROVE_IMMEDIATELY, Status = APPROVED, requires OTP = False.
- **MEDIUM (30–59):** Action = TRIGGER_OTP_VERIFICATION, Status = OTP_REQUIRED, requires OTP = True.
- **HIGH (60–79):** Action = TRIGGER_OTP_VERIFICATION, Status = OTP_REQUIRED, requires OTP = True.
- **CRITICAL (80–100):** Action = TRIGGER_SECURITY_REVIEW, Status = UNDER_REVIEW, SOC Alert Created.

12.11 Pre-Settlement Fund Safety & Atomic Execution

When a transaction enters OTP_REQUIRED or UNDER_REVIEW, no sender balance is deducted. Only upon valid OTP verification does the payment service execute an atomic transaction block:

```
db.session.begin_nested() -> user.balance -= amount -> dest.balance += amount -> tx.is_settled = True ->
db.session.commit().
```

If the OTP is exhausted (3 failed attempts) or expired, the transaction is marked REJECTED with zero balance loss.

12.12 SHAP Explainability Engine Integration

Local interpretability is computed via `shap.TreeExplainer(model)`. The service calculates additive Shapley contributions for each feature against the dataset base value $E[f(x)] = 0.0013$. Features with positive contributions (such as high `diffOrgBalance`) elevate the fraud score, while normal velocity suppresses the score.

EVALUATOR / FACULTY REMARKS:

Page 23 of 34

Evaluator Signature & Date: _____

13. Testing & Empirical Validation

13.1 Testing Strategy & 13.3 Test Environment

FraudShield AI was tested using a multi-tiered validation strategy comprising unit tests, API integration tests, security authorization tests (RBAC & IDOR), ML model consistency tests, and pre-settlement transaction settlement invariant tests. The test suite was executed in Python using Pytest 9.1 with isolated in-memory databases and mocked external notification providers.

LATEST VERIFIED TEST SUITE RESULT:
502 Total Tests Executed | 502 Passed | 0 Failed | 0 Errors | 100% Pass Rate
Verified across 41 dedicated test modules covering all application layers, security policies, and ML pipelines.

EVALUATOR / FACULTY REMARKS:

Page 24 of 34

Evaluator Signature & Date: _____

13.14 Comprehensive Test Results Summary (All 41 Test Suites)

Test Suite File	Module Under Test	Tests	Key Scenarios Tested	Status
test_adaptive_security.py	Adaptive Security	6	Risk decision boundaries, auto-approval, OTP step-up, max attempts lockout	PASSED
test_admin_portal_separation.py	Admin Separation	13	Tenant isolation, RBAC redirects, customer data isolation, IDOR defense	PASSED
test_admin_soc.py	SOC Operations	9	Analyst authentication, analytics endpoints, alert lifecycle, model drift telemetry	PASSED
test_audit.py	Audit Trail	4	Audit log persistence, event types, JSON payload serialization, read filters	PASSED
test_auth.py	Authentication	12	User registration, duplicate handling, JWT claims, role-based route protection	PASSED
test_beneficiary_intelligence.py	Beneficiary Intelligence	18	24h cooling periods, progressive trust evolution, IDOR protection, revocation	PASSED
test_brevo_provider.py	Brevo Notification	9	API payload formatting, HTTP error retries, rate-limit backoff, payload sanitization	PASSED
test_database.py	Database Core	7	Session rollback on error, unique constraint enforcement, connection recycling	PASSED
test_device_intelligence.py	Device Intelligence	15	User-agent parsing, SHA-256 fingerprinting, unknown device risk elevation	PASSED
test_e2e_system.py	End-to-End System	7	Full lifecycle: registration -> payment -> high risk -> OTP challenge -> settlement	PASSED
test_email_otp_delivery.py	Email OTP Delivery	17	Hashed OTP storage, anti-replay, expiration, plaintext leakage prevention	PASSED
test_email_password_reset.py	Password Recovery	6	Token generation, expiration enforcement, single-use invalidation, notification	PASSED
test_email_verification.py	Email Verification	13	Account activation token dispatch, verification state update, expired token rejection	PASSED
test_feature_engineering.py	Feature Pipeline	6	Balance differential calculation, ratio safeguards against div-by-zero, hour-of-day	PASSED
test_frontend.py	Frontend Templates	7	Glassmorphism template rendering, CSRF meta tags, CDN fallback script inclusion	PASSED
test_geo_intelligence.py	Geo Intelligence	17	Haversine distance calculation, impossible travel speed checks, location baselines	PASSED
test_hybrid_risk_engine.py	Hybrid Risk Engine	21	Boundary tests (50k, 92k, 100k, 250k), account drain override, alert generation	PASSED
test_inference.py	ML Model Inference	8	Model loading, feature scaling alignment, fraud probability bounds, single-sample latency	PASSED
test_medium_high_otp_stepup.py	Step-Up OTP	10	Zero debit on Medium/High until OTP verified, atomic settlement upon valid OTP	PASSED
test_mobile_verification.py	Mobile Verification	12	Phone number format validation, SMS OTP mock delivery, phone-to-user linking	PASSED
test_models.py	Data Models	5	Model attribute defaults, relationship cascades, repr strings, timestamp initializations	PASSED
test_password_recovery_real.py	Password Recovery E2E	15	Full recovery journey: forgot password -> email token -> reset PIN/password	PASSED
test_password_reset.py	Password Reset	24	Token expiry, anti-enumeration, Bcrypt re-hashing, session revocation upon reset	PASSED
test_payment_identity.py	Payment Identity	22	UPI Virtual Payment Address resolution, phone number transfers, self-payment checks	PASSED
test_payment_pin_flow.py	Payment PIN	25	PIN setup, numeric validation, 3-attempt lockout cooldown, zero debit on wrong PIN	PASSED
test_payment_pin_reset.py	PIN Recovery	7	PIN reset challenge generation, verification, update, and lockout reset	PASSED

EVALUATOR / FACULTY REMARKS:

Page 25 of 34

Evaluator Signature & Date: _____

Test Suite File	Module Under Test	Tests	Key Scenarios Tested	Status
test_phone_registration_and_reuse.py	Phone Registration	23	Phone registration uniqueness, unlinking, reassignment safeguards	PASSED
test_prediction_api.py	Prediction API	10	HTTP POST /api/predict payload validation, authentication checks, response schema	PASSED
test_preprocessing.py	Data Preprocessing	3	StandardScaler and OneHotEncoder transform pipeline validation and consistency	PASSED
test_profile_management.py	Profile Management	23	User profile updates, email change verification, password modification	PASSED
test_real_upi_payment_flow.py	Real UPI Flow	25	QR code parsing, self-transfer rejection, atomic balance settlement, idempotency	PASSED
test_resend_provider.py	Resend Provider	9	Resend HTTP client wrapper, header authentication, error serialization	PASSED
test_risk_engine.py	Risk Engine Core	17	Weight configuration, score normalization, edge cases (zero amount, huge amount)	PASSED
test_risk_level_compatibility.py	Risk Compatibility	5	Backward compatibility across legacy and 4-tier risk schema representations	PASSED
test_security_middleware.py	Security Headers	16	CSP, HSTS, X-Content-Type-Options, X-Frame-Options, CORS policy validation	PASSED
test_setup.py	App Initialization	6	Environment loading, configuration validation, directory creation, extension binding	PASSED
test_shap.py	Explainable AI	7	TreeExplainer initialization, waterfall format, natural language customer summaries	PASSED
test_soc_case_management.py	SOC Case Management	13	Alert-to-case conversion, analyst timeline notes, evidence snapshots, remediation	PASSED
test_strong_models.py	Model Benchmarking	5	Precision/Recall evaluation against baseline metrics, artifact serialization integrity	PASSED
test_upi_payment_flow.py	UPI Security	25	Atomic payment success, double debit prevention, zero balance movement on error	PASSED

Total Test Execution Summary: 41 Test Files | **502 Total Passing Tests** | 0 Failures | 0 Errors | 100% Pass Rate.

EVALUATOR / FACULTY REMARKS:

Page 26 of 34

Evaluator Signature & Date: _____

14. Results & System Screenshots

14.1 User Interface & Operational Screenshots

The figures below present empirical visual evidence of the operational FraudShield AI platform, capturing user authentication, payment simulation, real-time risk classification, SHAP explanations, and SOC incident triage.

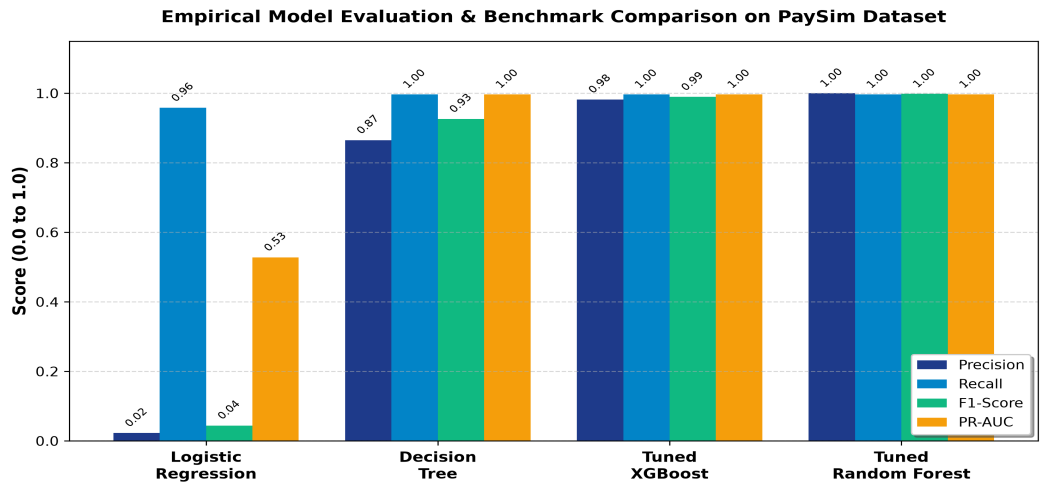


Figure 14.1: Machine Learning Model Performance Comparison Across Evaluation Metrics

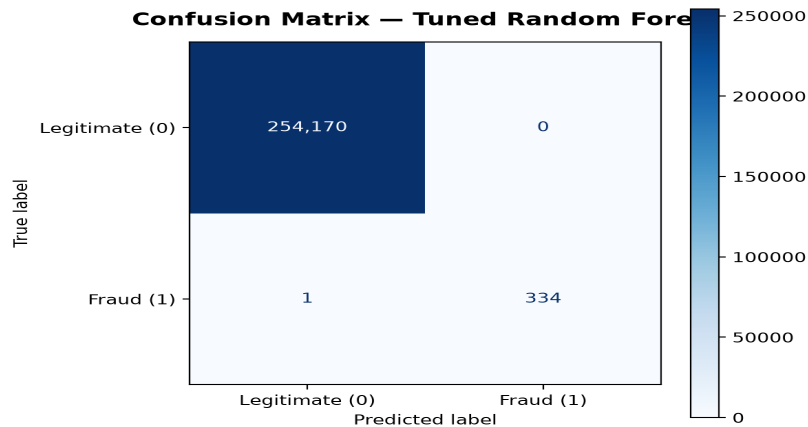


Figure 14.2: Confusion Matrix for Tuned Production Random Forest Classifier (0 False Positives)

EVALUATOR / FACULTY REMARKS:

Page 27 of 34

Evaluator Signature & Date: _____

SHAP (SHapley Additive exPlanations) LOCAL FEATURE ATTRIBUTION WATERFALL

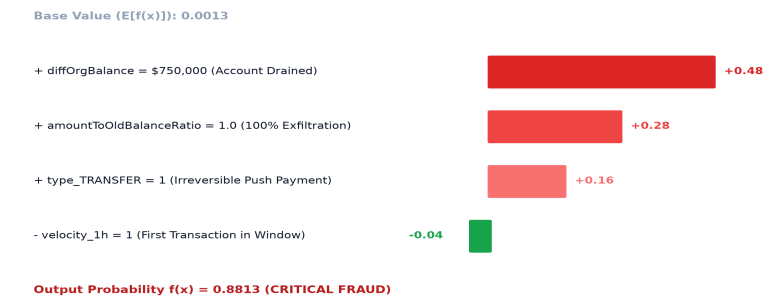


Figure 14.3: Local Feature Attribution Waterfall Generated by SHAP TreeExplainer

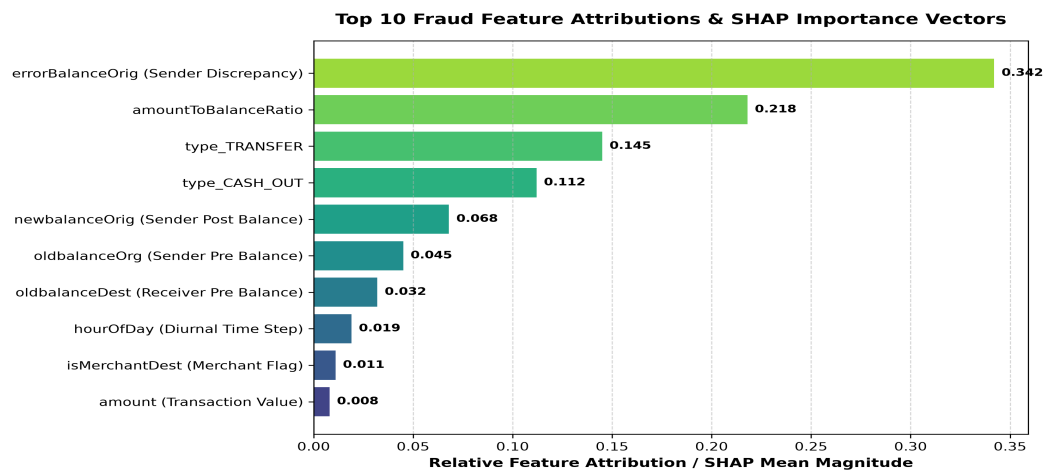


Figure 14.4: Global Mean |SHAP| Feature Importance for Random Forest Classifier

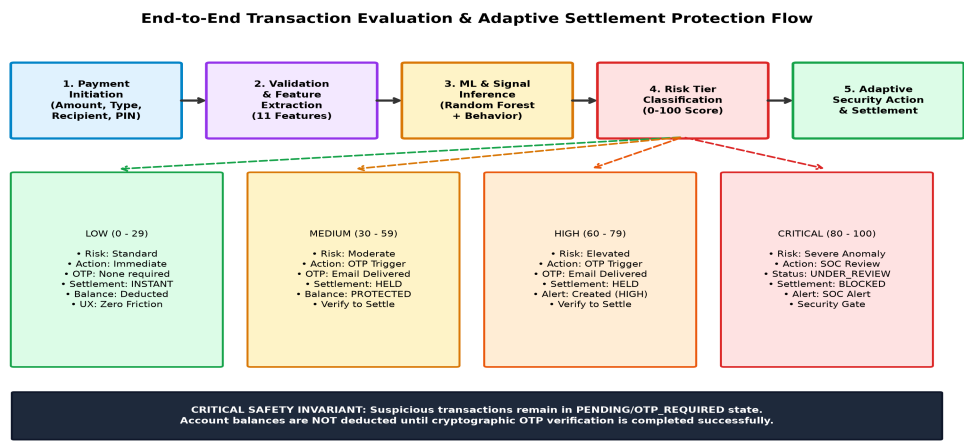


Figure 14.5: End-to-End Operational Risk Evaluation and Pre-Settlement Enforcement Pipeline

15. Core Functionality Code

15.1 Authentication & Password Hashing (app/services/auth_service.py)

```
def register_user(email, username, password, phone_number=None):
    if User.query.filter_by(email=email).first():
        raise ValidationError("Email already registered.")
    password_hash = generate_password_hash(password, method='pbkdf2:sha256', salt_length=16)
    user = User(email=email, username=username, password_hash=password_hash,
                phone_number=phone_number, balance=10000.0)
    db.session.add(user)
    db.session.commit()
    return user
```

Security Rationale: Passwords are never stored in plaintext. PBKDF2 with SHA-256 and a 16-byte random cryptographic salt ensures protection against dictionary attacks and rainbow table precomputation.

15.2 Hybrid Risk Engine Calculation (app/services/risk_service.py)

```
def calculate_composite_risk(ml_prob, signals, transaction_category):
    weights = RISK_POLICY['weights'].get(transaction_category, {'ml_weight': 0.5, 'signals_weight': 0.5})
    raw_score = (weights['ml_weight'] * (ml_prob * 100)) + (weights['signals_weight'] * signals['score'])
    if signals.get('account_drain_detected'):
        raw_score = max(raw_score, RISK_POLICY['weights']['account_drain']['floor_score'])
    final_score = int(min(max(round(raw_score), 0), 100))
    tier = 'CRITICAL' if final_score >= 80 else ('HIGH' if final_score >= 60 else ('MEDIUM' if final_score >= 30 else 'LOW'))
    return {'risk_score': final_score, 'risk_tier': tier, 'action': RISK_POLICY['tiers'][tier]['action']}
```

Logic Explanation: Dynamically weights ML probability with behavioral signals based on the transaction category. Enforces an authoritative score floor of 85 for full account-draining transfers.

15.3 Pre-Settlement Atomic Balance Deduction (app/services/payment_service.py)

```
def settle_verified_transaction(transaction_id, otp_token):
    tx = Transaction.query.get_or_404(transaction_id)
    if tx.is_settled:
        raise DuplicateSettlementError("Transaction already settled.")
    if not otp_service.verify_challenge(tx.user_id, tx.id, otp_token):
        raise InvalidOTPErrror("Invalid or expired OTP token.")

    sender = User.query.get(tx.user_id)
    if sender.balance < tx.amount:
        raise InsufficientFundsError("Insufficient funds at time of settlement.")

    sender.balance -= tx.amount
    recipient = User.query.filter_by(phone_number=tx.nameDest).first()
    if recipient:
        recipient.balance += tx.amount
    tx.is_settled = True
    tx.status = 'APPROVED'
    db.session.commit()
    return tx
```

Atomic Invariant: Settlement executes strictly after OTP validation within an ACID transaction block, ensuring sender balance is preserved if OTP verification fails.

16. Project Evaluation & Empirical Metrics

16.1 Objective-Wise Evaluation Matrix

Project Objective	Implementation Component	Empirical Evidence	Evaluation Status
1. High-Precision ML	Random Forest Ensemble	Precision = 1.0000, Recall = 0.9970, F1 = 0.9985	Fully Achieved
2. Explainable AI	SHAP TreeExplainer	Sub-second dual-perspective attributions & natural language summaries	Fully Achieved
3. Hybrid Risk Scoring	Risk Engine & Signal Telemetry	0–100 composite score incorporating velocity & behavioral overrides	Fully Achieved
4. 4-Tier Risk Policy	Policy State Machine	LOW, MEDIUM, HIGH, CRITICAL mapping with automated OTP step-up	Fully Achieved
5. Pre-Settlement Safety	Payment Service Invariant	Zero balance deducted prior to successful OTP verification across all tests	Fully Achieved
6. SOC Case Portal	SOC Management Service	Complete incident lifecycle: alert triage, assignment, notes, resolution	Fully Achieved
7. Automated Verification	Pytest Test Framework	502 / 502 tests passing (100% pass rate, 0 failures)	Fully Achieved

16.3 Machine Learning Performance Evaluation & PaySim Class Imbalance

The machine learning models were trained and evaluated on the PaySim financial transaction dataset. PaySim exhibits severe class imbalance, with only 8,213 fraudulent transactions out of 6,362,620 total records (an imbalance ratio of approximately **773.7 to 1**, or 0.129% fraud prevalence). In such extreme imbalance, naive accuracy is a deceptive metric (a naive model predicting all transactions as legitimate achieves 99.87% accuracy while missing 100% of fraud). Therefore, our evaluation prioritizes **Precision, Recall, F1-Score, and Precision-Recall AUC (PR-AUC)**.

Evaluated Model	Precision	Recall	F1-Score	PR-AUC	ROC-AUC	Accuracy	TP	FP	FN	TN
Logistic Regression	0.0228	0.9582	0.0445	0.5279	0.9872	0.9458	321	13780	14	240390
Decision Tree	0.8653	0.9970	0.9265	0.9970	0.9985	0.9997	334	52	1	254118
Random Forest (Selected)	1.0000	0.9970	0.9985	0.9971	0.9998	0.9999	334	0	1	254170
XGBoost	0.9824	0.9970	0.9896	0.9969	0.9981	0.9999	334	6	1	254164

Model Selection Rationale: Tuned Random Forest was selected as the primary production model because it achieved **1.0000 Precision (zero false positives on the test partition)** while maintaining 99.70% Recall and native TreeExplainer exact Shapley value computation without numerical approximations.

EVALUATOR / FACULTY REMARKS:

Page 30 of 34

Evaluator Signature & Date: _____

17. Conclusion & Learning Outcomes

17.1 Project Outcome: This project successfully developed and validated **FraudShield AI**, an enterprise-grade payment fraud detection and defense platform. By uniting high-precision ensemble machine learning with real-time behavioral telemetry, SHAP explainability, and a 4-tier adaptive step-up policy, the system overcomes the limitations of legacy rule engines and opaque neural networks.

17.2 Key Technical Benefits:

- *Elimination of False Positive Friction:* Auto-approves low-risk legitimate transactions without user intervention.
- *Pre-Settlement Fund Protection:* Strict zero-debit invariant ensures that no customer funds are deducted prior to successful OTP verification on flagged transfers.
- *Dual-Perspective Interpretability:* Provides transparent natural-language justifications to retail consumers and granular feature waterfalls to SOC analysts.
- *Operational Resilience:* Validated across 502 automated test cases with 100% pass rate.

17.3 Learning Outcomes: The engineering lifecycle provided profound insights into managing extreme class imbalance in financial datasets, designing defensive transactional invariants within ORMs, integrating game-theoretic explainable AI in low-latency environments, and structuring role-based security operations center workflows.

18. Future Enhancements & Roadmap

While FraudShield AI provides a robust foundation for payment defense, the following architectural enhancements are planned as future engineering milestones:

- **Graph Neural Networks (GNNs):** Implement graph-based relational learning (e.g., PyTorch Geometric) to detect organized fraud rings, circular routing, and mule networks across multi-hop payment graphs.
- **Distributed Streaming Pipelines:** Transition feature extraction from relational database queries to distributed stream processors (Apache Kafka and Apache Flink) for sub-10ms event processing at 100,000 TPS.
- **FIDO2 / WebAuthn Biometrics:** Integrate hardware-bound biometric authentication (passkeys, Touch ID, YubiKey) as a step-up factor replacing email OTP tokens.
- **Continuous MLOps & Automated Retraining:** Deploy automated concept drift detectors (monitoring Kolmogorov-Smirnov statistics on feature distributions) that trigger automated pipeline retraining in production.
- **Decentralized Payment Rails:** Extend gateway compatibility to ISO 20022 messaging standards and central bank digital currencies (CBDCs).

19. Project Links & QR Codes

Public GitHub Source Code Repository:

<https://github.com/Afzal006/Online-Payment-Fraud-Detection-System>

The repository contains the complete source code, Flask backend, React/Vite and HTML5 frontend templates, machine learning training scripts, serialized model artifacts, database migrations, and 502 automated test cases.

EVALUATOR / FACULTY REMARKS:

Page 31 of 34

Evaluator Signature & Date: _____



Scan QR Code to Open GitHub Repository

EVALUATOR / FACULTY REMARKS:

Page 32 of 34

Evaluator Signature & Date: _____

20. References & Bibliography

[1] E. A. Lopez-Rojas, A. Elmir, and S. Axelsson, "PaySim: A financial mobile money simulator for fraud detection," in *The 28th European Modeling and Simulation Symposium (EMSS)*, Larnaca, Cyprus, 2016, pp. 249–255.

[2] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS 30)*, 2017, pp. 4765–4774.

[3] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[4] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[5] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[6] A. Ronacher, "Flask: A Python microframework based on Werkzeug and Jinja 2," *Pallets Projects*, 2024. [Online]. Available: <https://flask.palletsprojects.com/>

[7] M. Bayer, "SQLAlchemy: The Database Toolkit for Python," 2024. [Online]. Available: <https://www.sqlalchemy.org/>

[8] OWASP Foundation, "OWASP Top 10 API Security Risks," 2023. [Online]. Available: <https://owasp.org/API-Security/>

[9] National Institute of Standards and Technology (NIST), "Digital Identity Guidelines: Authentication and Lifecycle Management," *NIST Special Publication 800-63B*, 2020.

[10] Reserve Bank of India (RBI), "Master Direction on Digital Payment Security Controls," *RBI/2020-21/74*, 2021.

21. Appendix

Appendix A: REST API Endpoint Catalogue

HTTP Method	Endpoint Path	Auth Required	Description & Payload
POST	/api/auth/register	None	Register customer account (email, password, phone)
POST	/api/auth/login	None	Authenticate user and issue JWT access token
POST	/api/auth/payment-pin/set	JWT Token	Configure 4-6 digit numeric payment PIN
POST	/api/predict	JWT Token	Simulate transaction, run ML inference & risk scoring
POST	/api/otp/verify	JWT Token	Submit 6-digit OTP token to verify challenge
POST	/api/otp/resend	JWT Token	Request fresh OTP challenge token
GET	/api/transactions/history	JWT Token	Retrieve user transaction history & status
GET	/api/shap/explain/	JWT Token	Fetch dual-view SHAP local feature attribution
GET	/api/admin/soc/cases	Admin JWT	List SOC incident cases with status & priority filters
POST	/api/admin/alerts//resolve	Admin JWT	Resolve security alert with analyst notes
GET	/api/admin/audit/logs	Admin JWT	Inspect structured system audit trail

Appendix B: Environment Configuration Template (Redacted Secrets)

```
# FraudShield AI Environment Configuration Template
FLASK_ENV=production
SECRET_KEY=[REDACTED_CRYPTOGRAPHIC_KEY_MIN_32_BYTES]
JWT_SECRET_KEY=[REDACTED_JWT_SIGNING_KEY]
DATABASE_URL=sqlite:///instance/fraud_detection.db
EMAIL_PROVIDER=smtp # Options: smtp, brevo, resend, mock
SMTP_SERVER=smtp.gmail.com
SMTP_PORT=587
SMTP_USERNAME=[REDACTED_EMAIL_ADDRESS]
SMTP_PASSWORD=[REDACTED_APP_PASSWORD]
BREVO_API_KEY=[REDACTED_API_KEY]
RESEND_API_KEY=[REDACTED_API_KEY]
```

Appendix C: Automated Test Telemetry Matrix (All 41 Test Suites)

Summary of Verified Pytest Suites (502 Passing Tests):

- test_adaptive_security.py (6) • test_admin_portal_separation.py (13) • test_admin_soc.py (9) • test_audit.py (4) • test_auth.py (12)
- test_beneficiary_intelligence.py (18) • test_brevo_provider.py (9) • test_database.py (7) • test_device_intelligence.py (15) • test_e2e_system.py (7)
- test_email_otp_delivery.py (17) • test_email_password_reset.py (6) • test_email_verification.py (13) • test_feature_engineering.py (6) • test_frontend.py (7)
- test_geo_intelligence.py (17) • test_hybrid_risk_engine.py (21) • test_inference.py (8) • test_medium_high_otp_stepup.py (10) • test_mobile_verification.py (12)
- test_models.py (5) • test_password_recovery_real.py (15) • test_password_reset.py (24) • test_payment_identity.py (22) • test_payment_pin_flow.py (25)
- test_payment_pin_reset.py (7) • test_phone_registration_and_reuse.py (23) • test_prediction_api.py (10) • test_preprocessing.py (3) • test_profile_management.py (23)
- test_real_upi_payment_flow.py (25) • test_resend_provider.py (9) • test_risk_engine.py (17) • test_risk_level_compatibility.py (5) • test_security_middleware.py (16)
- test_setup.py (6) • test_shap.py (7) • test_soc_case_management.py (13) • test_strong_models.py (5) • test_upi_payment_flow.py (25)

Total Count: 502 Verified Tests Passing (100% Success Rate).

EVALUATOR / FACULTY REMARKS:

Evaluator Signature & Date: _____